

HDOC Day-30

Menu-Driven Array Operations in C

Question

Write a C program to perform the following menu-driven operations on a given array:

1. Sort the elements using Bubble Sort algorithm and perform a Binary Search.
2. Perform Insertion and Deletion in a given valid position.
3. Rotate the array elements to the right by k positions.
4. Exit.

Algorithm

1. Start.
2. Read the number of elements n and the array elements.
3. Display the menu and read the user choice.
4. If choice is 1, sort the array using Bubble Sort, display it, read the search key and perform Binary Search.
5. If choice is 2, read an element and a valid insertion position, insert it, then read a valid deletion position and delete that element.
6. If choice is 3, read k and rotate the array to the right by k positions.
7. If choice is 4, terminate the program.
8. For any other choice, display an invalid-choice message.
9. Repeat the menu until Exit is selected.
10. Stop.

Test Case Table

Test	Operation	Input details	Expected output
1	Bubble Sort + Binary Search	n=5; 12 5 8 1 9; key=8	Sorted: 1 5 8 9 12; key 8 at position 3
2	Insertion + Deletion	10 20 30 40 50; insert 25 at pos 3; delete pos 5	After insert: 10 20 25 30 40 50; after delete: 10 20 25 30 50
3	Right Rotation	1 2 3 4 5; k=2	4 5 1 2 3
4	Exit	7 8 9; choice=4	Program terminated.

C Program

```
#include <stdio.h>
#define MAX 100

void display(int a[], int n) {
    int i;
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}

void bubbleSort(int a[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j]; a[j] = a[j + 1]; a[j + 1] = temp;
            }
        }
    }
}

int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] == key) return mid;
        else if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main() {
    int a[MAX], n, choice, i, key, pos, value, k, index, temp[MAX];
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);

    do {
        printf("\n----- MENU ----- \n");
        printf("1. Bubble Sort and Binary Search\n");
        printf("2. Insertion and Deletion\n");
        printf("3. Rotate array to the right by k positions\n");
        printf("4. Exit\n");
    }
```

C Program (continued)

```
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {
    case 1:
        bubbleSort(a, n);
        printf("Sorted array: "); display(a, n);
        printf("Enter element to search: "); scanf("%d", &key);
        index = binarySearch(a, n, key);
        if (index != -1) printf("Element %d found at position %d\n", key, index + 1);
        else printf("Element %d not found\n", key);
        break;
    case 2:
        printf("Enter element to insert: "); scanf("%d", &value);
        printf("Enter insertion position (1 to %d): ", n + 1); scanf("%d", &pos);
        if (pos < 1 || pos > n + 1) { printf("Invalid insertion position\n"); break; }
        for (i = n; i >= pos; i--) a[i] = a[i - 1];
        a[pos - 1] = value; n++;
        printf("Array after insertion: "); display(a, n);
        printf("Enter deletion position (1 to %d): ", n); scanf("%d", &pos);
        if (pos < 1 || pos > n) { printf("Invalid deletion position\n"); break; }
        for (i = pos - 1; i < n - 1; i++) a[i] = a[i + 1];
        n--;
        printf("Array after deletion: "); display(a, n);
        break;
    case 3:
        printf("Enter k: "); scanf("%d", &k);
        if (n > 0) k = k % n;
        for (i = 0; i < n; i++) temp[(i + k) % n] = a[i];
        for (i = 0; i < n; i++) a[i] = temp[i];
        printf("Array after right rotation: "); display(a, n);
        break;
    case 4:
        printf("Program terminated.\n");
        break;
    default:
        printf("Invalid choice.\n");
}
} while (choice != 4);
return 0;
}
```

Compilation, Execution & Testing

Test Case 1 - Bubble Sort and Binary Search

```
Desktop — zsh — 80x24

puppie@Anvikas-MacBook-Air Desktop % clang hdoc30.c -o hdoc30
puppie@Anvikas-MacBook-Air Desktop % ./hdoc30
Enter number of elements: 5
Enter 5 elements: 12 5 8 1 9

----- MENU -----
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
Enter your choice: 1
Sorted array: 1 5 8 9 12
Enter element to search: 8
Element 8 found at position 3

----- MENU -----
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
Enter your choice: 4
Program terminated.
puppie@Anvikas-MacBook-Air Desktop %
```

Test Case 2 - Insertion and Deletion

```
puppie@Anvikas-MacBook-Air Desktop % ./hdoc30
Enter number of elements: 5
Enter 5 elements: 10 20 30 40 50
```

```
----- MENU -----
```

1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit

```
Enter your choice: 2
```

```
Enter element to insert: 25
```

```
Enter insertion position (1 to 6): 3
```

```
Array after insertion: 10 20 25 30 40 50
```

```
Enter deletion position (1 to 6): 5
```

```
Array after deletion: 10 20 25 30 50
```

```
----- MENU -----
```

1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit

```
Enter your choice: 4
```

```
Program terminated.
```

```
puppie@Anvikas-MacBook-Air Desktop %
```

Test Case 3 - Right Rotation

```
Desktop — zsh — 80x24

puppie@Anvikas-MacBook-Air Desktop % ./hdoc30
Enter number of elements: 5
Enter 5 elements: 1 2 3 4 5

----- MENU -----
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
Enter your choice: 3
Enter k: 2
Array after right rotation: 4 5 1 2 3

----- MENU -----
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
Enter your choice: 4
Program terminated.
puppie@Anvikas-MacBook-Air Desktop %
```

Test Case 4 - Exit

```
Desktop — zsh — 80x24

puppie@Anvikas-MacBook-Air Desktop % ./hdoc30
Enter number of elements: 3
Enter 3 elements: 7 8 9

----- MENU -----
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
Enter your choice: 4
Program terminated.
puppie@Anvikas-MacBook-Air Desktop %
```

Result: The menu-driven C program for Bubble Sort with Binary Search, insertion and deletion, right rotation, and exit was successfully implemented and tested against the prepared test cases.